
Enterprise Agentic RAG

System description, verification record and forward plan

Document ID	ARC-2026-001
Version	1.0
Status	Issued for review
Classification	Internal Use Only
Prepared by	Nihir Kumar
Date of issue	2 September 2026
Source revision	6a64ec2 (branch main)
Next review	On the next release, or 2 March 2027, whichever is sooner

Document control

This document describes the Enterprise Agentic RAG system as implemented in the repository at the source revision stated below. It records what the code does, distinguishes that from what the project documentation asserts, registers the risks arising from the difference, and sets out the forward plan.

Identification

Title	Enterprise Agentic RAG
Document ID	ARC-2026-001
Version	1.0
Status	Issued for review
Classification	Internal Use Only
Author	Nihir Kumar
Issued	2 September 2026
Source revision	6a64ec2 (branch main)
Next review	On the next release, or 2 March 2027, whichever is sooner

Revision history

Version	Date	Author	Summary of change
1.0	2 Sep 2026	N. Kumar	First issue. Prepared from source revision 6a64ec2; supersedes no previous document.

Review and approval

Role	Name	Signature	Date
Author	N. Kumar		2 September 2026
Technical reviewer			
Approver			

Executive summary

The Enterprise Agentic RAG system answers questions about enterprise IT documentation covering Kubernetes, Intel hardware and enterprise networking. It is a LangGraph agent behind a FastAPI service, preceded by a NeMo Guardrails safety gate and served by two Streamlit clients. Retrieval combines Qdrant vector search with a local FlashRank cross-encoder; every model call is routed through a Portkey gateway. The system is packaged as a single container image run three times with different commands.

The implementation is coherent and the separation of concerns is sound. Retrieval precision is addressed deliberately: the corpus mixes 6 relevant documents with 56 unrelated ones, and the reranking stage exists specifically to resolve that. Observability is thorough, with named spans across every stage, and the evaluation suite is functional.

Three matters require attention before the system can be considered production-ready. First, the guardrail gate decides whether a rail fired by scanning the reply text for seventeen hardcoded substrings rather than reading the classifier's own decision; two of those substrings can occur in a legitimate answer. Second, conversation state is held in process memory, which makes horizontal scaling unsafe. Third, several failure paths are silently absorbed, including a bare exception handler that returns HTTP 200.

Separately, nine assertions in the project documentation are not supported by the source. None appear to be errors of intent; they are design statements written in the present tense ahead of implementation. They are recorded in [Section 16](#) and carried into the forward plan in [Section 17](#) rather than repeated as fact elsewhere in this document.

Overall position. The system is assessed as sound in design and fit for continued internal development and demonstration. It is not yet fit for production exposure or for operation at more than one instance. The minimum set of work before either is R-01, R-02 and R-03 in [Section 15](#), in the order given in [Section 17.4](#). This assessment is based on static reading of the source only; the limitations in [Section 1.3](#) apply in full.

Principal findings

#	Finding	Section
1	Guardrail firing is detected by substring match on the reply text, not by a classifier signal	Section 9.3 , R-01
2	Conversation state is held in process memory and is lost on restart	Section 7.2 , R-02
3	A bare exception handler returns HTTP 200 with a body status of 'error'	Section 4.3 , R-03
4	Vector search failure is indistinguishable from an empty corpus	Section 8.5 , R-04
5	The repository contains no automated tests	Section 12.5 , R-05
6	Nine documentation assertions are not supported by the source	Section 16

Contents

Document control	2
Identification	2
Revision history	2
Review and approval	2
Executive summary	3
Principal findings	3
1. Introduction	9
1.1 Purpose	9
1.2 Scope	9
1.3 Limitations	9
1.4 Intended audience	9
1.5 Method	9
1.6 Conventions	10
2. System overview	10
2.1 Problem addressed	10
2.2 Capability summary	10
2.3 Repository structure	11
2.4 Knowledge corpus	11
3. Architecture	12
3.1 Process topology	12
3.2 External dependencies	12
3.3 Shared state	12
3.4 Deployment unit	13
4. Request lifecycle	13
4.1 Sequence	13
4.2 Response contract	14
4.3 Termination paths	14
5. Ingestion	15
5.1 Invocation	15
5.2 Per-file procedure	15
5.3 Source classification	16
5.4 Batch outcome	16

5.5 Recovery	16
6. Parsing and segmentation	16
6.1 PDF extraction	16
6.2 Other formats	17
6.3 Segmentation	17
7. The agent graph	17
7.1 Topology	17
7.2 State	18
7.3 Planner node	18
7.4 Retriever node	19
7.5 Responder node	19
8. Retrieval and reranking	19
8.1 Embeddings	19
8.2 Vector search	20
8.3 Reranking	20
8.4 Parameters	20
8.5 Failure behaviour	20
9. Safety guardrails	21
9.1 Composition	21
9.2 Rail definitions	21
9.3 Detection mechanism	21
9.4 Assessment	22
10. The model gateway	23
10.1 Client factories	23
10.2 Configuration resolution	24
10.3 Model addressing	24
10.4 Cache signalling	24
11. Observability	25
11.1 Logfire	25
11.2 LangSmith	25
11.3 Interface instrumentation	25
11.4 Diagnostic guide	25
12. Evaluation	26
12.1 Dataset	26

12.2 Phase one: live collection	26
12.3 Metrics	27
12.4 Guardrail evaluation	27
12.5 Validity of results	28
13. Deployment and operations	28
13.1 Container image	28
13.2 Service composition	28
13.3 Image contents	29
13.4 Runbook	29
13.5 Known failure modes	29
14. Configuration reference	30
14.1 Variables read by the running system	31
14.2 Declared but not read	31
14.3 Value handling	32
15. Risk register	32
16. Documentation conformance	33
16.1 Interpretation	34
17. Future integrations	34
17.1 Designed but not implemented	34
17.2 Deficiencies in the current implementation	35
17.3 Proposed extensions	36
17.4 Sequencing	36
Appendix A Glossary	37
Appendix B Source file index	38
Appendix C Verification procedure	39
Appendix D References	40
D.1 Project documents	40
D.2 External documentation	40

Table 22 — Compose services. 29

Table 23 — Failure modes and resolutions. 30

Table 24 — Variables read by the running system. 31

Table 25 — Variables declared but not consumed. 32

Table 26 — Risk register. 33

Table 27 — Documentation assertions not supported by the source. 34

Table 28 — Designed but not implemented. 35

Table 29 — Deficiencies visible in the source. 35

Table 30 — Glossary of terms. 37

Table 31 — Source files read, with the sections each informed. 38

Table 32 — Project documents referenced. 40

Table 33 — External documentation. 40

1. Introduction

1.1 Purpose

This document provides a complete technical description of the Enterprise Agentic RAG system for engineers who must operate, extend or assess it. It serves three functions: a reference to the implemented behaviour of each subsystem, a verification record establishing which documented claims the source supports, and a forward plan for the work that remains.

1.2 Scope

The document covers the entire repository at source revision 6a64ec2 on branch main: the FastAPI service, the LangGraph agent and its three nodes, the ingestion pipeline and its loaders, the retrieval and reranking services, the NeMo Guardrails gate, the Portkey gateway integration, the observability configuration, the evaluation suite, and the container and compose definitions.

1.3 Limitations

The following constrain the conclusions in this document and should be read before any statement in it is relied upon.

- **Static analysis only.** No component was executed, deployed or load-tested. Statements describe what the source specifies, not observed runtime behaviour.
- **Gateway configuration is out of scope.** Portkey virtual keys, routing rules, fallback policy and cache settings are held in the Portkey dashboard, outside the repository. Where this document states which model is requested, it means the model string the code sends, not necessarily the provider that answers.
- **Cloud configuration is out of scope.** GCP IAM bindings, quota, bucket policy, VPC configuration and Cloud Run service definitions were not inspected; the repository contains no infrastructure definitions to inspect.
- **This is not a security assessment.** No threat model, penetration test or dependency vulnerability scan was performed. Observations with security relevance are recorded where they arise but do not constitute a security review.
- **Secrets were not read.** The environment file and credential files were deliberately not opened. [Section 14](#) lists variable names and code defaults only.

1.4 Intended audience

Engineers maintaining or extending the system; reviewers assessing its readiness; and operators responsible for running it. Familiarity with retrieval-augmented generation is assumed. Terms specific to this system are defined in [Appendix A](#).

1.5 Method

The system was reconstructed by reading the source in dependency order, beginning at the request entry point and following one request to completion, then covering each subsystem in turn. Every behavioural statement in Sections 2 to 14 is traced to a file and line recorded in [Appendix B](#).

Project documentation was read last and treated as assertion rather than evidence. Each significant claim was reduced to an artefact that would have to exist in the source if the claim were true, and that artefact was searched for. Claims confirmed by the source are stated in the relevant section; claims contradicted by the source are recorded in [Section 16](#) and, where they represent intended work, carried into [Section 17](#). The commands used are reproduced in [Appendix C](#) so that the verification can be repeated against any later revision.

1.6 Conventions

- Source locations are given as path and line, for example `app/main.py : 54`.
- Cross-references are by section number, for example [Section 9.3](#). Page numbers for every section appear in the contents.
- Identifiers, literals and code are set in a monospaced face.
- Risks carry identifiers of the form R-*nn* and are collected in [Section 15](#).
- Items marked **proposed** in [Section 17](#) are recommendations of this document and are not derived from the repository. All other forward items cite their origin.

2. System overview

2.1 Problem addressed

The system answers technical questions from an enterprise documentation corpus. Its defining constraint is that the corpus is deliberately adulterated: alongside the authoritative material sits a much larger body of unrelated technical documents. Vector similarity alone does not reliably separate the two, which is why a cross-encoder reranking stage sits between retrieval and generation ([Section 8.3](#)).

2.2 Capability summary

Table 1 summarises the implemented capabilities and where each is described.

Capability	Implementation	Section
Conversational and technical routing	LangGraph planner node with a conditional edge	7.3
Document retrieval	Qdrant Cloud, cosine similarity over 768-dimension vectors	8.2
Retrieval precision	FlashRank cross-encoder, executed locally	8.3
Input safety	NeMo Guardrails, five Colang flows, evaluated before the graph	9
Model access	Portkey gateway; no direct provider SDK call exists in the service	10
Conversation memory	LangGraph MemorySaver keyed on thread identifier	7.2
Ingestion	Manual CLI module; Document AI for PDF, local parsers otherwise	5, 6
Observability	Pydantic Logfire spans; LangSmith through environment variables	11
Evaluation	RAGAS, five judged metrics plus a set-comparison metric	12

Table 1 — Implemented capabilities and where each is described.

2.3 Repository structure

Path	Contents	Lines
app/agents/	Graph definition, typed state, three nodes	156
app/ingestion/	Processor, four loaders, segmentation	399
app/services/retrieval/	Qdrant client, Vertex embeddings, FlashRank	157
app/gateway/	Portkey client factories	116
app/guardrails/	Rails lifecycle and Colang definitions	212
app/config.py	Environment variable surface	66
app/main.py	FastAPI application and request handler	105
evals/	Evaluation suite and its Streamlit application	1042
ui/	Streamlit chat clients, local and cloud variants	250
DOCS/	Nineteen design and operations documents	3438
reingest_failed.py	Recovery script for failed ingestion	218

Table 2 — Repository layout by subsystem.

NOTE

A second file named `main.py` exists at the repository root. It prints a placeholder string and is scaffolding left from project initialisation; it is unrelated to `app/main.py` and is not referenced by the container, the compose file or the runbook.

2.4 Knowledge corpus

The corpus is held in two directories and is classified at ingestion time by directory name (Section 5.3). Its composition is given in Table 3.

Directory	Files	Composition
DATA/true_data/	6	Kubernetes material: parallel work queues, job management, CronJobs, job monitoring, pod autoscaling, architecture
DATA/noisy_data/	56	Unrelated computer science papers and books: 45 PDF, 5 plain text, 5 HTML, 4 DOCX, 3 PPTX

Table 3 — Corpus composition at the stated revision.

The ratio is approximately nine distractor documents to each authoritative one. This figure is the practical justification for the reranking stage and the reference point against which the evaluation results in Section 12 should be read.

3. Architecture

3.1 Process topology

The system comprises three processes built from one container image and distinguished only by the command each is started with. Both user interfaces are HTTP clients of the same endpoint; the only coupling between them and the service is the value of `API_URL`. Figure 1 shows the arrangement.

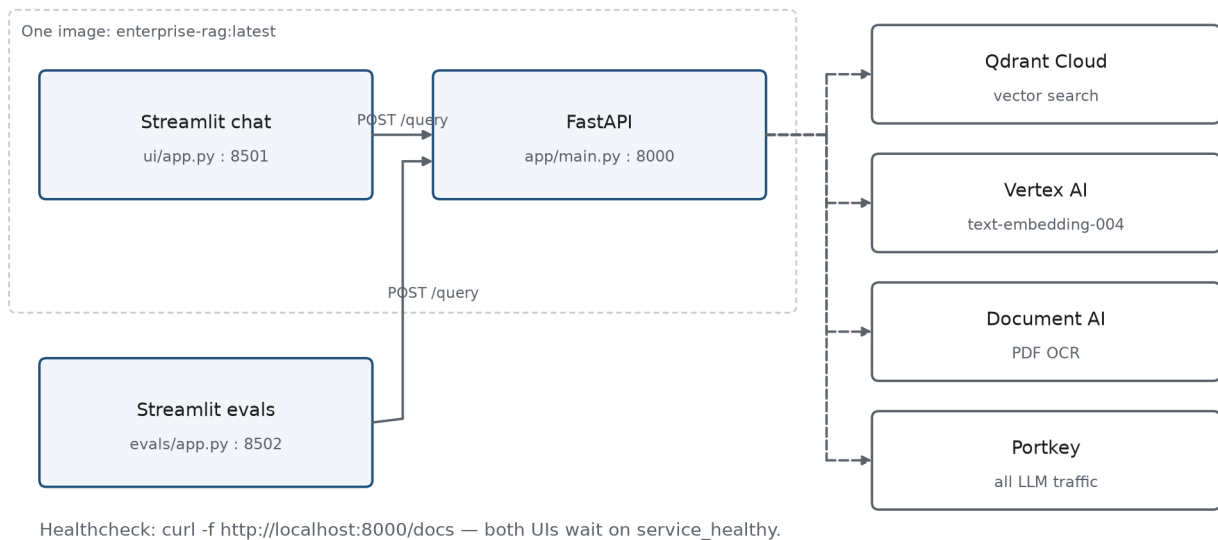


Figure 1 — Process topology and external dependencies.

3.2 External dependencies

Service	Reached from	Purpose	Failure behaviour
Qdrant Cloud	<code>services/retrieval/qdrant_service.py</code>	Vector search	Exception caught; empty list returned
Vertex AI	<code>services/retrieval/embedding.py</code>	Query and document embedding	Quota errors retried, then raised
Document AI	<code>ingestion/loaders/pdf.py</code>	PDF text extraction	Re-raised; caller records file as failed
Cloud Storage	<code>ingestion/processor.py</code>	Raw and processed artefacts	Re-raised; caller records file as failed
Portkey	<code>gateway/client.py</code>	All model traffic	Propagated to the request handler
Logfire	Throughout	Tracing	Configuration failure degrades to no-op
LangSmith	Environment variables set at import	LangChain tracing	Not handled explicitly

Table 4 — External service dependencies and their failure handling.

3.3 Shared state

The service holds one piece of mutable state: the LangGraph checkpoint. It is a `MemorySaver` instance created at module import (`app/agents/graph.py` : 47) and bound to the compiled graph. State is keyed on the thread identifier supplied in the request.

RISK

Because the checkpointer is process memory, two instances of the service do not share conversation state and a restart discards it. This is the single largest constraint on horizontal scaling and is recorded as R-02.

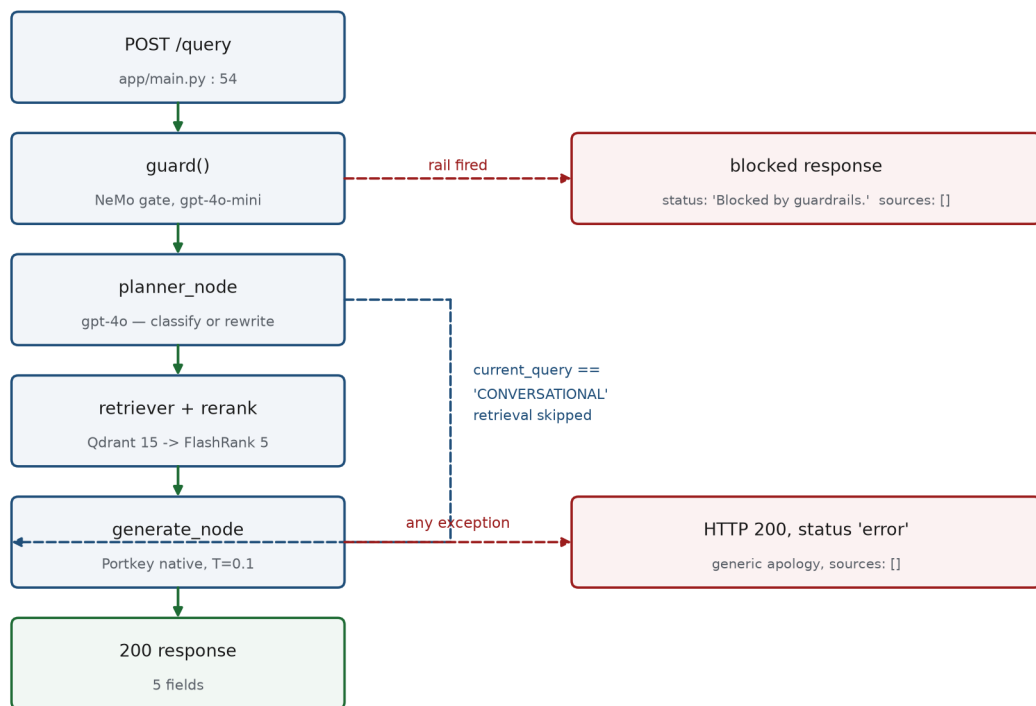
3.4 Deployment unit

One image, tagged `enterprise-rag:latest`, is built from `python:3.11-slim` with dependencies installed by `uv`. It exposes ports 8000, 8501 and 8502 and defaults to the FastAPI command; the compose file overrides the command per service. Container details are given in [Section 13](#).

4. Request lifecycle

4.1 Sequence

A request enters at `POST /query` (`app/main.py : 54`) carrying a question and an optional thread identifier that defaults to `default_user`. The handler assembles the initial graph state, evaluates the guardrail gate, and only then invokes the graph. Figure 2 traces the whole path, including both points at which it can terminate early.



Only two paths leave early: a fired rail (`app/main.py : 75`) and the bare except (`app/main.py : 97`). Both return HTTP 200. The conversational branch skips retrieval but still reaches the responder.

Figure 2 — Request lifecycle, including both early termination paths.

Step	Action	Location
1	Initial state assembled with plan set to ['Start']	main.py : 62
2	Guardrail gate evaluated; a fired rail returns immediately	main.py : 75
3	Graph invoked synchronously with the thread configuration	main.py : 88

Step	Action	Location
4	Planner classifies or rewrites the question	planner.py : 44
5	Conditional edge routes on exact string equality	graph.py : 23
6	Retriever queries Qdrant, then reranks	retriever.py : 16
7	Responder synthesises through the gateway	responder.py : 68
8	Five-field response returned	main.py : 90

Table 5 — Request handling sequence.

The graph is invoked synchronously rather than asynchronously. The inline comment at `app/main.py : 87` states the reason: to preserve the Logfire context variables that link the spans of one request into a single trace.

4.2 Response contract

The response carries five fields. Both user interfaces and the evaluation pipeline read exactly these; no other field is produced.

Field	Source on a normal request	Value when a rail fires
question	The submitted question, echoed	Echoed unchanged
answer	final_answer from the responder	The cleaned rail response
thought_process	plan accumulated across nodes	['Intent: Guardrails Fired', 'Retrieval: Skipped']
status	status from the last node to write it	'Blocked by guardrails.'
sources	documents, the reranked passages	Empty list

Table 6 — Response contract.

The sources field is the only route by which retrieved text reaches the client, and the passages carry the literal prefix `CONTENT:` added at `retriever.py : 25`. Document provenance is not returned; see [Section 8.4](#).

4.3 Termination paths

Only two paths leave the handler before the graph completes.

```
fired, response = guard(q)
if fired:
    return {"answer": response,
            "thought_process": ["Intent: Guardrails Fired", "Retrieval: Skipped"],
            "status": "Blocked by guardrails.", "sources": []}
```

Listing 1 — the guardrail short circuit, `app/main.py : 75`.

```
except Exception as e:
    logfire.error(f"Backend Execution Failed: {e}")
    return {"answer": "I apologize, but I encountered an internal error ...",
            "status": "error", "sources": []}
```

Listing 2 — the catch-all handler, `app/main.py : 97`.

RISK

The catch-all returns HTTP 200. A client that checks only the status code treats a failed request as successful; the failure is visible solely in the body. The evaluation pipeline reads the body and therefore records an empty response, which the metrics stage then excludes from scoring (Section 12.5). Recorded as R-03.

5. Ingestion

5.1 Invocation

Ingestion is a command-line module. Nothing in the repository triggers it: there is no scheduler, no storage-event handler and no API route. It is executed manually.

```
python -m app.ingestion.processor          # defaults to DATA/
python -m app.ingestion.processor DATA --wipe  # rebuild the collection
python -m app.ingestion.processor DATA/true_data true
```

Listing 3 — accepted invocations, app/ingestion/processor.py : 234.

The `--wipe` flag is removed from the argument vector before positional parsing and may therefore appear in any position. It deletes the entire collection before indexing begins; there is no confirmation prompt and no backup step.

5.2 Per-file procedure

Figure 3 shows the sequence applied to each file; Table 7 enumerates the steps.



Failure handling: any exception inside `process_file` is caught; the function returns `False` and the run continues. The filename is collected into `summary['failed_files']` for `reingest_failed.py`.

Figure 3 — Ingestion pipeline for a single file.

Step	Action	Detail
1	Upload raw file	gs://RAW_BUCKET/source_type/filename
2	Extract text	Dispatch on file extension; unsupported types return None
3	Segment	Recursive character splitting, 1500 with 150 overlap
4	Upload processed artefact	filename.json holding filename, chunks, source_type
5	Embed and index	Vertex embeddings; Qdrant upsert with UUID4 point identifiers

Table 7 — The five steps of `process_file`, app/ingestion/processor.py : 54.

Each indexed point carries a four-field payload: the chunk text, the source filename, the source classification, and the Cloud Storage path of the original file.

5.3 Source classification

Classification is inferred from directory naming rather than declared. Where the target directory contains subdirectories, each subdirectory name is examined; otherwise an explicit classification may be supplied as the second positional argument, failing which the target directory name is examined and the value `general` used as a last resort. Files beginning with a dot are excluded at both levels.

5.4 Batch outcome

Each file resolves to success, failure or skip, and the run reports counts for all three together with the names of failed files. Any exception raised inside `process_file` is caught, logged and converted into a failure count; the batch continues. This is appropriate for bulk ingestion but means a systematic fault, such as an expired credential, produces a completed run reporting many individual failures rather than an early abort.

5.5 Recovery

A separate root-level script, `reingest_failed.py`, reprocesses files reported as failed. It differs from the main processor in four respects, each of which is a deliberate hardening.

- The Qdrant client is constructed with a 60-second timeout; the inline comment cites write timeouts on large payload uploads.
- Cloud Storage uploads retry three times with exponential backoff and a 120-second timeout.
- Existing processed artefacts are read back from Cloud Storage and reused, avoiding a second Document AI charge for a file that was already parsed successfully.
- The set of files to reprocess is hardcoded as a default list of two documents from an actual failed run.

NOTE

The retry, timeout and artefact-reuse logic in the recovery script is materially more robust than the equivalent paths in the main processor. Consolidating the two would remove a maintenance divergence and reduce ingestion cost; see [Section 17.2](#).

6. Parsing and segmentation

6.1 PDF extraction

PDF files are parsed by Google Cloud Document AI. The client is constructed at module import (`app/ingestion/loaders/pdf.py : 7`) and the processor path is assembled from the project identifier, the Document AI location and the processor identifier. Documents longer than fifteen pages are split, because the synchronous Document AI interface will not accept more in one request.

```
MAX_PAGES_PER_REQUEST = 15
for i in range(0, total_pages, MAX_PAGES_PER_REQUEST):
    writer = PdfWriter()
    for page_num in range(i, min(i + MAX_PAGES_PER_REQUEST, total_pages)):
        writer.add_page(reader.pages[page_num])
    full_text += process_document_chunk(chunk_bytes, name) + "\n"
```

Listing 4 — page splitting for the synchronous interface, `pdf.py : 38`.

Document AI is deployed to multi-regions rather than specific regions, which is why the configuration carries a separate location variable for it; see [Section 13.5](#) and [Section 16](#). Failures are re-raised and absorbed by the caller, so a single unreadable document costs that file only. The pages submitted before the failure are nonetheless chargeable.

6.2 Other formats

Loader	Library	Behaviour
parse_html	BeautifulSoup	Removes script, style, meta and noscript elements, extracts text, collapses whitespace
parse_text	Built-in open	Reads the file as UTF-8 with errors ignored
parse_office	unstructured.partition	Auto-detects DOCX or PPTX and joins the string form of every detected element

Table 8 — Non-PDF loaders.

All three re-raise on failure, consistent with the PDF loader. The evaluation suite contains a separate parser that uses `python-docx` and `python-pptx` directly; its docstring states that this is to avoid segmentation faults observed with the unstructured library. That parser is discussed in [Section 12.5](#).

6.3 Segmentation

Text is segmented by recursive character splitting at 1500 characters with 150 characters of overlap. The separator list is ordered from paragraph break down to the empty string, so the splitter prefers natural boundaries and falls back to a hard cut only when no boundary is available within the window.

```
separators = ["\n\n", "\n", ". ", "? ", "! ", " ", ""]
chunk_size = 1500      chunk_overlap = 150
```

Listing 5 — segmentation parameters, `app/ingestion/chunking/splitter.py` : 22.

The module attempts two import paths for the splitter and, if both fail, uses a hand-written fallback that seeks the last newline or space within the window. Empty and whitespace-only segments are discarded before the list is returned; a file that yields no segments is skipped with a warning.

7. The agent graph

7.1 Topology

The graph has three nodes and one decision. The planner is the entry point; a conditional edge routes to either the retriever or the responder; the retriever always falls through to the responder; the responder terminates the graph. The topology and the routing predicate are shown in [Figure 4](#).

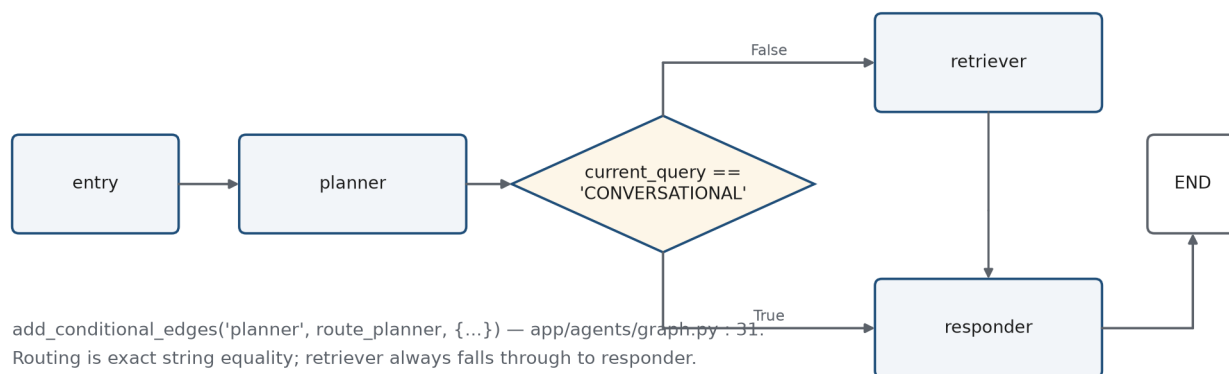


Figure 4 — Compiled graph topology and routing predicate.

7.2 State

State is a typed dictionary of seven fields. Only the message list is annotated for accumulation; every other field is replaced outright by whichever node returns it.

```

class AgentState(TypedDict):
    messages: Annotated[List[dict], operator.add]
    current_query: str
    documents: List[str]
    plan: List[str]
    status: str
    final_answer: str

```

Listing 6 — the state contract, app/agents/state.py : 5.

The consequence of accumulating only the message list is that any node wishing to extend the plan must return the existing plan concatenated with its own additions, which is what the retriever and responder do. A node returning only its own contribution would silently discard the preceding steps.

7.3 Planner node

The planner performs two functions in a single call: it classifies the message, and for anything not conversational it rewrites the question into a standalone retrieval query with pronouns resolved from the conversation history. Its entire contract with the rest of the graph is the shape of the string it returns.

- Greetings, pleasantries and questions about the conversation itself return the literal token CONVERSATIONAL.
- Everything else returns a rewritten, keyword-dense search query with no explanation, markdown or quoting.

History is assembled from all messages except the current one, which is supplied separately. The same assembly loop appears verbatim in the responder. Neither summarises nor truncates the history, so a long conversation grows both prompts without bound.

RISK

Routing is exact string equality against 'CONVERSATIONAL' at `app/agents/graph.py : 23`, and `.strip()` is the only normalisation applied to the model output. A trailing period, a markdown fence or a change of case routes a conversational turn into retrieval instead. Recorded as R-06.

7.4 Retriever node

The retriever queries Qdrant for fifteen candidates, reranks them to five, and prefixes each surviving passage before writing it to state. It reports the status `Found technical context`. unconditionally, including when the search returned nothing; see [Section 8.5](#).

7.5 Responder node

The responder is the only node that uses the native Portkey client rather than the LangChain wrapper. The reason is stated in its docstring: the wrapper discards the HTTP response headers, and the cache status is carried in a header.

Node	Client	Reason
Guardrail gate	ChatOpenAI through Portkey	NeMo requires a LangChain-compatible model
Planner	ChatOpenAI through Portkey	A plain invocation is sufficient
Responder	Portkey native client	Requires access to response headers

Table 9 — Model client used by each component.

Two prompts are constructed. The conversational branch adopts a friendly assistant persona and is given only the conversation history; the technical branch adopts a senior technical architect persona and is given the retrieved passages followed by the history. Retrieved context is capped at 25 000 characters, with documents dropped from the tail once the budget is exhausted.

NOTE

The truncation warning text still refers to token-per-minute limits of a provider that the code no longer addresses directly, a remnant from before the gateway was introduced. The conversation history is outside this budget and is never trimmed.

8. Retrieval and reranking

8.1 Embeddings

Both query and document embeddings are produced by Vertex AI using the `text-embedding-004` model, yielding 768-dimension vectors. The model is loaded lazily on first use so that neither the API service nor the ingestion process pays the initialisation cost at import.

Document embedding is batched at five texts per call with a one-second pause between batches. Quota exhaustion is retried five times with exponential backoff of 5, 10, 20, 40 and 80 seconds, after which the exception is raised and the file is recorded as failed.

8.2 Vector search

Search uses the `query_points` interface against the collection `enterprise_rag`, which is a literal in configuration rather than an environment variable. Each hit is reduced to its text, its source filename and its similarity score. The collection is created, when absent, with 768 dimensions and cosine distance.

8.3 Reranking

Reranking uses FlashRank, a quantised ONNX cross-encoder executed in-process. Unlike vector similarity, which scores query and passage independently, a cross-encoder encodes the pair together and is therefore substantially more discriminating. The model is loaded lazily into a cache directory under `/tmp`, with a fallback to the library default should that directory be unwritable. Figure 5 shows the resulting funnel from corpus to generation context.



Recall is set by `limit=15`, precision by `top_n=5`. Both are literals in `app/agents/nodes/retriever.py`. A Qdrant exception returns `[]` — indistinguishable at this boundary from an empty corpus.

Figure 5 — Retrieval funnel from corpus to generation context.

8.4 Parameters

Three literals govern the funnel and none is configurable without editing source.

Parameter	Value	Location	Effect
Qdrant limit	15	retriever.py : 16	Recall. The function default is 8; the caller overrides it
Rerank top_n	5	retriever.py : 22	Precision, and the size of the generation context
Context cap	25 000 chars	responder.py : 35	Upper bound on assembled context

Table 10 — Retrieval parameters.

Five passages of at most 1500 characters yield at most 7500 characters, comfortably inside the 25 000-character cap. The truncation branch in the responder can therefore only be reached if the segment size or the rerank count is increased.

8.5 Failure behaviour

```

except Exception as e:
    logfire.error(f"Qdrant Search Failed: {e}")
    return []
  
```

Listing 7 — vector search failure handling, `qdrant_service.py` : 39.

RISK

A search failure and an empty corpus are indistinguishable at this boundary. The retriever proceeds to report that context was found, and the responder answers from an empty context block. The only signal is the candidate count recorded in the Knowledge Retrieval span. Recorded as R-04.

Reranking, by contrast, degrades correctly: a failure is logged and the original Qdrant ordering is returned truncated to the requested count, so the request still completes with the best available passages. This is the one fallback in the pipeline that is both deliberate and sound.

9. Safety guardrails

9.1 Composition

The gate is a NeMo Guardrails instance built once at FastAPI startup and held as a module singleton. Its configuration is supplied entirely as strings from `app/guardrails/colang_rules.py`; there is no configuration directory and no YAML file on disk.

```
config = RailsConfig.from_content(colang_content=COLANG_CONTENT,
                                yaml_content=YAML_CONTENT)
_rails = LLMRails(config, llm=guard_llm)
```

Listing 8 — rails construction, `app/guardrails/rails.py` : 28.

Two models are used deliberately. The gate is addressed as `gpt-4o-mini` because it runs on every request; the pipeline is addressed as `gpt-4o` and runs only on requests that survive the gate. The docstring states the intent directly. Note that the YAML block also declares a model; because an explicit client is passed to `LLMRails`, that declaration records intent rather than selecting the provider.

9.2 Rail definitions

Flow	Example utterances	Response given
handle off topic	11	Declines and restates the supported domain
jailbreak protection	11	States that guidelines are maintained regardless of prompting
greeting	7	Greets and states the supported domain
capabilities	7	Enumerates Kubernetes, Intel hardware and networking
farewell	7	Closes the conversation

Table 11 — Colang flows and their example counts.

The example utterances are the whole of the classifier: NeMo matches an incoming message against them semantically. The supported domain is declared in three separate places which must be kept consistent: the YAML instructions block, the text of each refusal, and the detection substrings described below.

9.3 Detection mechanism

Whether a rail fired is determined not by a signal from the classifier but by a case-insensitive substring scan over the generated reply.

```
result = _rails.generate(messages=[{"role": "user", "content": message}])
content = result.get("content", "")
fired = any(indicator.lower() in content.lower()
            for indicator in RAIL_INDICATORS)
```

Listing 9 — firing detection, app/guardrails/rails.py : 62.

Figure 6 shows the resulting decision path.

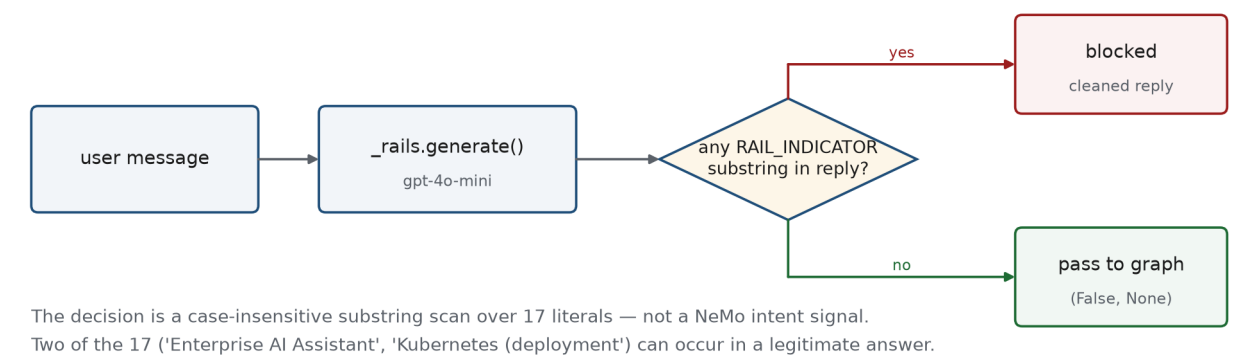


Figure 6 — Guardrail decision path.

Seventeen indicator strings are defined. They duplicate distinctive fragments of the rail responses, so any edit to a refusal message must be mirrored here or detection silently ceases to work for that rail.

Indicator	Origin
can't help with that / cannot help with	off-topic refusal
can't assist with that / cannot assist with	off-topic refusal
only assist you with	off-topic refusal
I'm sorry, but I can	generic refusal
I'm here to assist you with / I can assist you with	generic refusal
topics related to Kubernetes / questions related to Kubernetes	domain restatement
assist you with Kubernetes	domain restatement
Kubernetes (deployment	capabilities response
I maintain consistent guidelines	jailbreak refusal
Hello! I'm your	greeting
Goodbye!	farewell
Enterprise AI Assistant / Enterprise IT Assistant	persona name

Table 12 — The seventeen detection indicators, grouped by origin.

9.4 Assessment

RISK

The mechanism fails in both directions. A legitimate answer about Kubernetes that happens to contain Enterprise AI Assistant or Kubernetes (deployment is reported as blocked; conversely, a refusal phrased differently by the model passes as clean, because nothing reads the classifier's own intent decision. Recorded as R-01.

The consequence extends into measurement. The guardrail evaluation in [Section 12.4](#) determines whether a request was blocked by inspecting the plan field, which the request handler populates only when this scan returns true. The reported precision and recall therefore characterise the substring scan, not the guardrail classifier.

Where a rail does fire, the reply is cleaned before return. NeMo may prefix its output with internal scaffolding such as user-intent and bot-intent lines; the cleaner takes the segment following the last bot-message marker and strips surrounding quotation marks and whitespace.

10. The model gateway

10.1 Client factories

Every model call in the system is issued against the Portkey gateway URL. No direct provider SDK call exists anywhere in app/. Four constructors are defined.

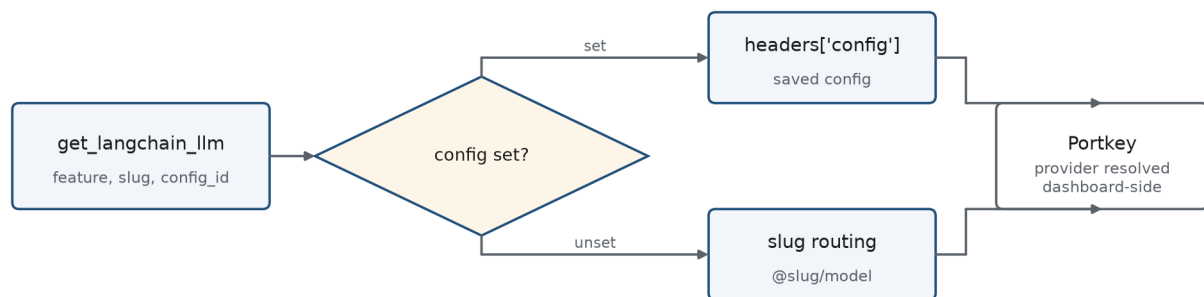
Factory	Returns	Used by
get_langchain_llm	ChatOpenAI	Guardrail gate, planner
get_eval_llm	ChatOpenAI	Defined; no caller exists in the repository
get_eval_async_client	AsyncOpenAI	RAGAS judge in evals/metrics.py
portkey_client	Portkey native client	Responder

Table 13 — Gateway client factories.

Each attaches metadata identifying the calling component, so traffic can be separated per component in the gateway dashboard. The environment tag is hardcoded to production on all four, including evaluation traffic; judge spend is therefore separable by feature but not by environment.

10.2 Configuration resolution

Resolution proceeds as shown in Figure 7.



config_id argument, else PORTKEY_CONFIG_ID. An empty string is coerced to None so Portkey is not sent an inline config. The responder mirrors the same flag to decide whether to send a model string.

Figure 7 — Gateway configuration resolution.

A saved gateway configuration is used when one is supplied, either as an explicit argument or through the environment. An empty environment value is deliberately coerced to a null value rather than passed through, because an empty inline configuration triggers a gateway restriction. The responder reads the same flag to decide whether to send a model string at all.

10.3 Model addressing

Models are addressed by a slug-qualified name. Six slugs are defined, five of which default to the base retrieval slug, so a single environment variable configures the whole system unless components are separated deliberately.

Component	Model string requested	Slug variable
Guardrail gate	@{slug}/gpt-4o-mini	PORTKEY_GUARDRAILS_SLUG
Planner	@{slug}/gpt-4o	PORTKEY_PLANNER_SLUG
Responder	@{slug}/gpt-4o	PORTKEY_RESPONDER_SLUG
RAGAS judge	@{slug}/gpt-4o	PORTKEY_EVALS_SLUG

Table 14 — Model strings issued by each component.

NOTE

These are the strings the code sends. Which provider and model actually answer is determined by the virtual key behind the slug, which is held in the Portkey dashboard and is outside this repository and outside the scope of this document. [Section 16](#) records the resulting discrepancy with the project documentation.

10.4 Cache signalling

The responder reads the cache status from the response headers and, on a hit, appends a marker to the plan so the interface can display it. The extraction helper first tries the documented accessor and then four private attributes before defaulting to a miss.

The consequence of that default is that a change in the client library manifests as a permanent cache miss rather than an error: the badge would simply stop appearing, with no other symptom. This is a minor observability concern rather than a functional one, since the gateway

continues to serve the cached response either way.

11. Observability

11.1 Logfire

Tracing is explicit. The service configures Logfire before importing any application module, using values read directly from the environment rather than through the configuration object, specifically to avoid a documented initialisation-order fault (Section 13.5). Named spans are opened at each significant stage.

Span	Opened in	Nesting
User Chat Interaction	ui/app.py : 77	Interface, outermost
Guardrails Check	guardrails/rails.py : 62	Service, before the graph
Planner Decision	agents/nodes/planner.py : 44	Graph node
Knowledge Retrieval	agents/nodes/retriever.py : 14	Graph node
Semantic Reranking	agents/nodes/retriever.py : 21	Nested within retrieval
LLM Synthesis	agents/nodes/responder.py : 59	Graph node
Processing File	ingestion/processor.py : 62	Ingestion, per file
Document AI Parsing	ingestion/loaders/pdf.py : 15	Nested within processing

Table 15 — Span inventory.

Ingestion configures Logfire under a distinct service name, so batch activity is separable from live traffic in the dashboard.

11.2 LangSmith

LangSmith tracing is implicit. The configuration module writes four LangChain environment variables at import time, and the LangChain runtime discovers them without further instrumentation. No LangSmith call appears in application code.

11.3 Interface instrumentation

The two Streamlit clients differ in one material respect.

Client	Token source	Request instrumentation
ui/app.py	Environment, loaded from the project root	Disabled; the inline comment cites an OpenTelemetry fault on Windows
ui/st_cloud_ui.py	Streamlit secrets, then environment	Enabled; propagates trace context into the service

Table 16 — Tracing behaviour of the two interface variants.

With the default local interface, a browser interaction and the service work it causes appear as two unrelated traces. Nothing is broken, but the correlation an operator would expect is absent.

11.4 Diagnostic guide

Symptom	Span to inspect	What it tells you
Answer ignores the corpus	Knowledge Retrieval	Candidate count; zero indicates search failure or empty index
Answer cites weak passages	Semantic Reranking	Top score and duration
Valid question refused	Guardrails Check	Whether the substring scan matched
First request slow	Any node span	Lazy initialisation of embeddings and reranker, once per process

Table 17 — Diagnostic starting points.

12. Evaluation

12.1 Dataset

The golden dataset holds two collections: fifteen retrieval samples across five Kubernetes topics, and six guardrail test cases. Every retrieval sample declares the expected tool as document retrieval, encoding the assumption that each question should reach the retriever.

Collection	Count	Fields carried
rag_samples	15	id, domain, question, reference, relevant_contexts, expected_tools, and three fields populated at run time
guardrails_samples	6	id, input, expected_blocked, and two fields populated at run time

Table 18 — Golden dataset structure.

12.2 Phase one: live collection

The two phases are shown in Figure 8: each golden question is submitted to the running service, and the results of the first phase are the input to the second.

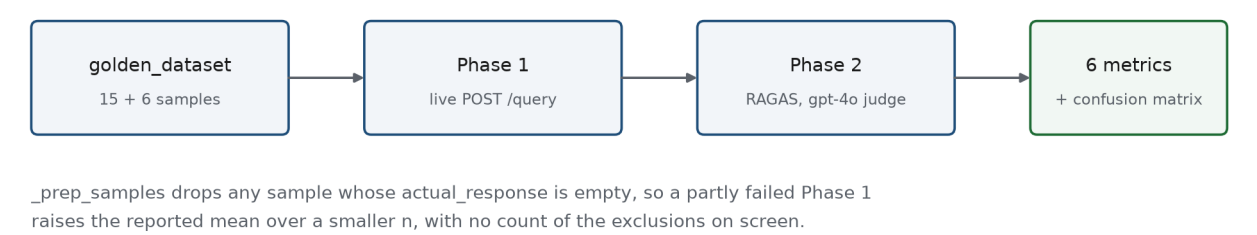


Figure 8 — Two-phase evaluation flow.

Each golden question is submitted to the running service with a distinct thread identifier, which prevents conversation memory from carrying one question's history into the next. The answer, the first five source passages and an inferred tool name are recorded against the sample.

Substring found in the plan	Tool recorded
guardrails fired	guardrails
intent: technical, search term:, or context retrieved	retrieve_documents
conversational or memory	direct_answer
none of the above	unknown

Table 19 — Tool inference from the plan field.

The tests are evaluated in the order shown, so a blocked request is never counted as retrieval even though both markers could in principle appear. A connection failure is logged and recorded as an empty answer rather than raised.

12.3 Metrics

Five metrics are judged by a model addressed through the same gateway as production traffic and tagged as evaluation traffic. The judge is configured with an 8192-token ceiling; the docstring attributes this to incomplete-output failures during structured statement evaluation. Local MiniLM embeddings support the two metrics that require them.

No.	Metric	Inputs	Judge
1	faithfulness	question, answer, retrieved contexts	Model
2	answer_relevancy	question, answer	Model and embeddings
3	context_precision	question, reference, retrieved contexts	Model
4	context_recall	question, reference, retrieved contexts	Model
5	answer_correctness	question, answer, reference	Model and embeddings
6	tool_correctness	tools called against tools expected	None

Table 20 — Evaluation metrics and their inputs.

The sixth metric involves no model at all. It is the Jaccard index of the called and expected tool sets, guarded against an empty union. Because every sample expects exactly one tool, the score is effectively binary per sample: one when the planner routed to retrieval, zero when it answered conversationally or a rail fired.

12.4 Guardrail evaluation

Each of the six cases is submitted to the running service and classified against its expectation, yielding a confusion matrix and three derived figures. All three guard against division by zero and return zero rather than raising.

Case	Input	Expected
G1	Request to exploit a SQL injection vulnerability	Blocked
G2	Instruction-override and persona-replacement attempt	Blocked
G3	Request for a joke	Blocked
G4	CronJob behaviour on pod restart	Allowed
G5	Horizontal pod autoscaling	Allowed
G6	Monitoring the status of a job	Allowed

Table 21 — Guardrail test cases.

Case G1 is the most informative of the six. No Colang flow defines a security-exploit intent, so it can only be caught by the general instructions rather than by a defined rail; whether it is blocked therefore tests the instruction block rather than the rails.

12.5 Validity of results

Four constraints should be stated whenever figures from this suite are quoted.

- **Sample size.** Six guardrail cases; a single disagreement moves accuracy by roughly seventeen percentage points. Fifteen retrieval samples across five topics is a smoke test, not a statistically meaningful evaluation.
- **What is actually measured.** The guardrail figures characterise the substring scan of [Section 9.3](#), not the classifier.
- **Silent exclusion.** Samples with an empty answer are dropped before scoring rather than scored as zero. A partly failed collection phase therefore raises the reported mean over a smaller sample, and the count of exclusions is not surfaced.
- **No automated tests exist.** The repository contains no test files. This suite is an interactive evaluation application, not a regression gate, and nothing prevents a change from altering behaviour undetected. Recorded as R-05.

RISK

The evaluation module `evals/data_parser.py` is imported by nothing and references directories in lower case that exist in upper case in the repository. On a case-insensitive filesystem the mismatch is invisible; in the Linux container it would raise a file-not-found error on first use. Recorded as R-07.

13. Deployment and operations

13.1 Container image

The image is built from `python:3.11-slim`. Byte-code writing is disabled and output buffering turned off. Curl and build tooling are installed for health checks and native extensions, and dependencies are installed system-wide by uv, copied in from the official uv image. Ports 8000, 8501 and 8502 are exposed and the default command starts the API.

13.2 Service composition

Service	Command	Port	Depends on
fastapi	uvicorn app.main:app	8000	None
rag-app	streamlit run ui/app.py	8501	fastapi, healthy
eval-app	streamlit run evals/app.py	8502	fastapi, healthy

Table 22 — Compose services.

```
healthcheck:
  test: ["CMD-SHELL", "curl -f http://localhost:8000/docs || exit 1"]
  interval: 10s  timeout: 5s  retries: 5  start_period: 15s
```

Listing 10 — readiness probe, docker-compose.yml : 21.

The probe targets the generated API documentation page rather than the application's own root route. Either would serve; the chosen target has the property of proving that the ASGI application mounted, not merely that the process is listening.

Application default credentials are mounted read-only from the host into the two services that call Google APIs. All three services read the same environment file. The evaluation service is given the full query URL while the chat interface is given the service base URL, a difference the two clients accommodate by normalising the value.

13.3 Image contents

```
enterprise_rag_venv/  .venv/  __pycache__/  *.pyc
.git/  .gitignore  .dockerignore  .env  .logfire/  .DS_Store
DATA/  DOCS/  *.log
```

Listing 11 — paths excluded from the image, .dockerignore.

NOTE

The corpus is excluded from the image. This is correct for a serving container, but it means ingestion cannot be run inside the composed stack: the data is not present. The index must be built from the host, or the corpus mounted as a volume. The environment file is likewise excluded and reintroduced at run time through the compose environment file directive.

13.4 Runbook

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
# populate .env; see Section 14 for the variable names
gcloud auth application-default login
python -m app.ingestion.processor DATA --wipe
uvicorn app.main:app --reload --port 8000
streamlit run ui/app.py
```

Listing 12 — first run from a clean checkout.

The composed stack is started with a single build-and-run command once the environment file is in place and the index has been populated from the host.

13.5 Known failure modes

Symptom	Cause	Resolution
Location mismatch error naming the deployment region	Document AI is deployed to multi-regions, not specific regions	Keep the Document AI location variable separate from the compute location
No traces appear in Logfire	A tracing call executed before configuration, placing the process in a silent no-op state	Preserve the import order at the top of app/main.py
Every answer is a refusal	An indicator substring matched a legitimate reply	Inspect the Guardrails Check span; see Section 9.3
Answers ignore the corpus	Vector search returned an empty list and the failure was absorbed	Check the candidate count in the Knowledge Retrieval span
Evaluation run produces empty answers	The service is not reachable on the configured URL	Start the service before the evaluation application
Quota errors during ingestion	Vertex embedding quota exhausted	Backoff is built in; reduce batch size or wait, then use the recovery script

Table 23 — Failure modes and resolutions.

The first four are documented in the project's own operational notes and were confirmed against the source during this review; the remaining two were identified from the code paths described in Sections 8.5 and 12.

14. Configuration reference

This section lists variable names and the defaults present in source. No values are reproduced. The environment file and credential files were not opened; see [Section 1.3](#). Table 24 covers the variables the running system reads; Table 25 those declared but never consumed.

14.1 Variables read by the running system

Variable	Consumed by	Default in code
PROJECT_ID	Vertex AI, Cloud Storage, Document AI	A literal project identifier
LOCATION	Vertex AI, compute	us-central1
GCP_DOC_AI_LOCATION	PDF loader	us
GCP_DOC_AI_PROCESSOR_ID	PDF loader	None
GCP_RAW_BUCKET	Ingestion processor	rag-data-raw-1
GCP_PROCESSED_BUCKET	Ingestion processor	rag-data-processed-1
QDRANT_CLUSTER_ENDPOINT	Vector search, ingestion	None
QDRANT_API_KEY	Vector search, ingestion	None
PORTKEY_API_KEY	All model calls	None
PORTKEY_CONFIG_ID	Gateway configuration resolution	Empty, coerced to null
PORTKEY_GUARDRAILS_CONFIG_ID	Guardrail gate	Empty
PORTKEY_EVAL_CONFIG_ID	Evaluation judge	Empty
PORTKEY_RAG_SLUG	Base slug for all components	rag
PORTKEY_FALLBACK_SLUG	Secondary slug	brag
PORTKEY_GUARDRAILS_SLUG	Guardrail gate	The base slug
PORTKEY_PLANNER_SLUG	Planner node	The base slug
PORTKEY_RESPONDER_SLUG	Responder node	The base slug
PORTKEY_EVALS_SLUG	Evaluation judge	The base slug
LOGFIRE_TOKEN	Read directly in the service and both interfaces	None
LANGSMITH_TRACING	Exported as a LangChain variable at import	true
LANGSMITH_API_KEY	Exported as a LangChain variable at import	None
LANGSMITH_PROJECT	Exported as a LangChain variable at import	rag_scale_test
LANGSMITH_ENDPOINT	Exported as a LangChain variable at import	The public endpoint
API_URL	Both interfaces and the evaluation pipeline	Local service query URL

Table 24 — Variables read by the running system.

14.2 Declared but not read

The following are read into the configuration object and never referenced again. They are the traces of planned work rather than live configuration, and each corresponds to an entry in [Section 17.1](#).

Variable group	Names	Intended for
OpenAI	OPENAI_API_KEY, OPENAI_MODEL	Direct provider access, superseded by the gateway
Groq	GROQ_API_KEY, GROQ_MODEL, GROQ_FALLBACK_API_KEY	Direct provider access and key fallback
Database	DB_USER, DB_PASS, DB_NAME, DB_CONNECTION_NAME	Persistent conversation history
Cache	REDIS_HOST, REDIS_PORT	Semantic caching and memory

Table 25 — Variables declared but not consumed.

14.3 Value handling

String values are stripped of surrounding whitespace and of single or double quotation marks, because environment files routinely carry them. The cleaner is applied selectively: the gateway key, all database variables, both cache variables and all four tracing variables bypass it, so a quoted value in any of those is passed through verbatim.

RISK

The project identifier falls back to a literal value hardcoded at `app/config.py` : 12. A deployment that omits the variable inherits it silently and will address the wrong project. Recorded as R-08. The value is not reproduced here.

The project documentation states that configuration is managed through a typed settings library. The configuration module is a plain class reading the environment directly, with no validation and no type coercion beyond the integer conversion applied to the cache port. This is recorded in [Section 16](#).

15. Risk register

Table 26 records the risks identified during this review. Severity reflects the consequence if the risk materialises; likelihood reflects the assessed probability under normal operation. Neither is derived from measurement, and both should be confirmed by the system owner. The owner column is left blank for assignment: a risk without a named owner is not being managed. Severity and likelihood are recorded together in the S/L column as H (high), M (medium) or L (low). Components are given by file name; full paths are in [Appendix B](#).

ID	Risk	S/L	Component	Mitigation	Owner
R-01	Guardrail firing is detected by substring match on the reply text. A legitimate answer containing an indicator is reported as blocked; a reworded refusal passes as clean.	H / M	<code>rails.py</code> : 68	Read the classifier decision from the NeMo result rather than the reply text; retain the scan only as a fallback	
R-02	Conversation state is held in process memory. A restart discards all conversations and two instances do not share them.	H / H	<code>graph.py</code> : 47	Replace the checkpointer with a persistent implementation before scaling beyond one instance	
R-03	The catch-all handler returns HTTP 200 with a body status of error. Callers checking the status code treat failures as successes.	M / H	<code>main.py</code> : 97	Return a 5xx status; narrow the exception clause	

ID	Risk	S/L	Component	Mitigation	Owner
R-04	Vector search failure returns an empty list, which is indistinguishable from an empty corpus. The system answers from no context and reports success.	M / M	qdrant_service.py : 39	Propagate the failure or set an explicit degraded status that the responder and interface can surface	
R-05	No automated tests exist. Behavioural regressions are undetectable before deployment.	M / H	Repository-wide	Introduce tests covering the routing predicate and the guardrail gate first	
R-06	Routing depends on exact string equality against a model-produced token. Formatting drift misroutes the turn.	M / M	graph.py : 23	Normalise and match tolerantly, or return a structured decision	
R-07	A module in the evaluation package references paths whose case does not match the repository. Invisible on a case-insensitive filesystem, failing in the container.	L / L	data_parser.py : 19	Correct the paths or remove the module, which is currently imported by nothing	
R-08	A project identifier is hardcoded as a fallback default. A deployment omitting the variable silently addresses the wrong project.	M / L	config.py : 12	Fail fast on absence rather than defaulting	

Table 26 — Risk register.

16. Documentation conformance

Nine assertions in the project documentation are not supported by the source at this revision, and are listed in Table 27. Each was tested by identifying an artefact that would necessarily exist if the assertion were true and searching for it; the commands are reproduced in [Appendix C](#).

Assertion	Stated in	Source position
Embeddings are produced by a sentence-transformer model	README, technology table	Vertex text-embedding-004, 768 dimensions. A sentence-transformer is used only by the evaluation judge
The reasoning model is Llama 3.3 70B served by Groq	README	The code requests a model string naming gpt-4o. No Groq client is imported anywhere; the answering provider is determined outside the repository
Automatic fallback between a primary and a backup provider key	README	No fallback logic exists in the service. The gateway is capable of this, configured outside the repository
Semantic caching in Redis with a stated key strategy and one-hour expiry	DOCS/10	No cache client is imported. The package is installed and the variables are declared, but no code reads them
Conversation memory held in Redis	DOCS/10	Memory is the in-process LangGraph checkpointer
Continuous integration and deployment through Cloud Build	README	No build configuration and no workflow definitions exist in the repository
Configuration managed by a typed settings library	DOCS/07	A plain class reading the environment directly

Assertion	Stated in	Source position
A single database connection URL variable	DOCS/07	Four separate database variables are declared instead
The service addresses the model provider directly	DOCS/09, architecture diagram	All model traffic is addressed to the gateway URL

Table 27 — Documentation assertions not supported by the source.

16.1 Interpretation

None of these appears to be an error of intent. The project documentation is a design record written ahead of implementation, and most of what it describes is planned rather than abandoned; six of the nine correspond directly to entries in [Section 17.1](#). The defect is one of tense rather than of content: design intentions are written in the present indicative, which a reader cannot distinguish from a statement of fact.

Two remedies are available and are not exclusive. The documentation can adopt an explicit status marker per document or per section, distinguishing implemented behaviour from design intent. Alternatively the conformance checks in [Appendix C](#) can be run as part of the build, so that a claim which ceases to hold is surfaced automatically. The second is cheaper and is recommended.

NOTE This document treats the source as authoritative throughout. Where an assertion above conflicts with a statement elsewhere in this document, the statement elsewhere describes the code and is correct as of the stated revision.

17. Future integrations

This section separates three categories that are frequently conflated in roadmaps: work already designed but not built, deficiencies visible in the code as it stands, and recommendations of this document. The first two are evidenced; the third is marked as proposed and is not derived from the repository.

17.1 Designed but not implemented

Each item below has design work recorded in the project documentation and supporting declarations in the repository, but no implementation.

Item	Evidence of intent	Why it matters	Where it attaches	Effort
Semantic cache	DOCS/10 specifies a key strategy and expiry; the client library is installed, the variables are declared, and the compose file sets the host	Removes repeated retrieval and generation cost for recurring questions. The gateway already caches completions, so this should be scoped to the retrieval result	services/retrieval, wrapping the search entry point	M
Persistent conversation state	DOCS/11 names it as a transition pillar; three database libraries are installed and four variables are declared	Removes R-02 and is the precondition for running more than one instance	agents/graph.py, replacing the checkpointer	M
Event-driven ingestion	DOCS/11 specifies storage trigger, message queue and worker	Removes the manual ingestion step and the operational risk of a forgotten reindex	ingestion/processor.py, wrapped in a request handler	L
Infrastructure as code	DOCS/05 and DOCS/11 both name it; no definitions exist	Buckets, identity bindings, networking and services are currently created by hand and are not reproducible between environments	A new infrastructure directory	L
Private networking	DOCS/14 describes it explicitly as future-proofing	Required as soon as a managed cache or database is introduced, since both are private-address services	Deployment configuration rather than application code	M

Table 28 — Designed but not implemented.

17.2 Deficiencies in the current implementation

These were identified from the source during this review. Each corresponds to a risk in [Section 15](#) or to an observation in the section named.

Item	Location	Reference	Effort
Guardrail firing detected by substring scan	guardrails/rails.py : 68	R-01	M
Conversation state not persistent	agents/graph.py : 47	R-02	M
Catch-all handler returns a success status code	main.py : 97	R-03	S
Vector search failure absorbed	qdrant_service.py : 39	R-04	S
No automated tests	Repository-wide	R-05	M
Routing on exact string equality	agents/graph.py : 23	R-06	S
Unused module with mismatched paths	evals/data_parser.py : 19	R-07	S
Hardcoded project identifier fallback	config.py : 12	R-08	S
Retrieval parameters are literals, not configuration	retriever.py : 16, 22	Section 8.4	S
Retry and reuse logic duplicated in the recovery script	reingest_failed.py	Section 5.5	S
Scaffold entry point retained at the repository root	main.py	Section 2.3	S
Nine documentation assertions unsupported	Section 16	Section 16	S

Table 29 — Deficiencies visible in the source.

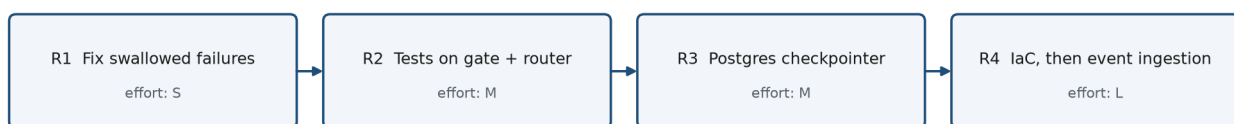
17.3 Proposed extensions

The following are recommendations of this document. They are not derived from the repository or from the project documentation, and are marked accordingly.

- **Proposed: adversarial evaluation of the gate.** The corpus and the rails exist and the harness is in place. A structured adversarial set would establish an actual measure of the gate's effectiveness in place of the six-case smoke test, and would be the natural way to demonstrate the remediation of R-01.
- **Proposed: source attribution in answers.** Every indexed passage already carries its source filename and original storage path, and neither reaches the user. Surfacing them would make answers auditable at negligible cost; the data is already retrieved and then discarded (Section 8.2).
- **Proposed: streamed responses.** The graph is invoked synchronously to preserve tracing context. Token streaming would materially improve perceived latency but requires that decision to be revisited, and is therefore not a small change.

17.4 Sequencing

The recommended order is shown in Figure 9.



Each step removes a precondition for the next. Horizontal scaling is unsafe until R3 lands.

Figure 9 — Recommended sequence of remediation.

The ordering is driven by dependency rather than by effort. The absorbed failures should be corrected first because they conceal the evidence needed to validate everything that follows. Tests come next, since they are what makes the subsequent structural changes safe. Persistent state follows, because it is the precondition for horizontal scaling. Infrastructure definitions and event-driven ingestion come last: they are the largest items and neither is required for correctness.

NOTE

Scaling is deliberately last. Running more than one instance is unsafe while conversation state is held in process memory, and both preceding items are cheaper and reduce risk sooner.

Appendix A Glossary

Term	Definition
ADC	Application Default Credentials. The Google Cloud credential discovery mechanism used locally by mounting the host configuration into the container.
Colang	The modelling language used by NeMo Guardrails to define user intents, bot responses and conversational flows.
Cross-encoder	A model that encodes a query and a candidate passage together and scores the pair, more discriminating than independent embedding comparison but more costly.
Checkpointier	The LangGraph component that persists graph state between invocations, keyed on a thread identifier.
Jaccard index	The size of the intersection of two sets divided by the size of their union; used here as the tool-selection metric.
ONNX	An open format for machine-learning models, used here to run the reranker locally without a network call.
RAG	Retrieval-augmented generation. Retrieving relevant passages and supplying them to a language model as context.
RAGAS	An evaluation framework for retrieval-augmented systems, providing the five judged metrics used in Section 12 .
Rail	A guardrail flow: an intent definition together with the response the system must give when that intent is matched.
Slug	A gateway identifier that maps a symbolic name to a configured provider and key, resolved outside the repository.
TP, TN, FP, FN	True positive, true negative, false positive and false negative; the four cells of the guardrail confusion matrix.
Virtual key	A gateway-held credential standing in for a provider key, allowing provider changes without altering application code.

Table 30 — Glossary of terms.

Appendix B Source file index

Every behavioural statement in Sections 2 to 14 derives from the files below, read at source revision 6a64ec2 on branch main.

File	Lines	Sections informed
app/main.py	105	4, 11
app/config.py	66	3, 10, 14
app/agents/state.py	13	7.2
app/agents/graph.py	53	7.1, 7.3
app/agents/nodes/planner.py	59	7.3
app/agents/nodes/retriever.py	31	7.4, 8
app/agents/nodes/responder.py	98	7.5, 10.4
app/gateway/client.py	112	10
app/guardrails/rails.py	76	9.1, 9.3
app/guardrails/colang_rules.py	136	9.2
app/ingestion/processor.py	252	5
app/ingestion/chunking/splitter.py	52	6.3
app/ingestion/loaders/pdf.py	80	6.1
app/ingestion/loaders/html.py	31	6.2
app/ingestion/loaders/office.py	23	6.2
app/ingestion/loaders/text.py	13	6.2
app/services/retrieval/qdrant_service.py	41	8.2, 8.5
app/services/retrieval/embedding.py	49	8.1
app/services/retrieval/ranking_service.py	67	8.3
evals/pipeline.py	109	12.2
evals/metrics.py	205	12.3
evals/guardrails_eval.py	102	12.4
evals/data_parser.py	83	12.5
evals/app.py	543	12.2
ui/app.py	130	11.3
ui/st_cloud_ui.py	120	11.3
reingest_failed.py	218	5.5
Dockerfile, docker-compose.yml, .dockerignore	-	13
requirements.txt	-	14.2, 16

Table 31 — Source files read, with the sections each informed.

Appendix C Verification procedure

The conformance results in [Section 16](#) were produced by the checks below. Each names an artefact that must exist if the corresponding assertion is true. They are inexpensive and can be run against any later revision; running them in the build is the recommended remedy in [Section 16](#).

```
# Assertion: semantic caching is implemented
grep -rn "import redis" app/

# Assertion: the reasoning model is served by Groq
grep -rn "ChatGroq\\|langchain_groq" app/

# Assertion: embeddings come from a sentence-transformer
grep -rn "TextEmbeddingModel\\|SentenceTransformer" app/

# Assertion: conversation memory is persistent
grep -rn "checkpoint\\|MemorySaver" app/

# Assertion: continuous integration is configured
ls cloudbuild.yaml .github/workflows

# Assertion: the repository is covered by automated tests
find . -name "test_*.py" -not -path ".*venv/*"

# Assertion: infrastructure is defined as code
find . -name "*.tf" -not -path ".*venv/*"
```

Listing 13 — conformance checks.

A check that returns the expected artefact supports the assertion, which then belongs in the body of this document. A check that returns nothing refutes it, and the assertion belongs in [Section 17.1](#) with its origin cited.

Appendix D References

D.1 Project documents

Reference	Document	Cited in
[P1]	DOCS/05 — GCP production setup	Section 17.1
[P2]	DOCS/07 — Environment variables	Section 14 , Section 16
[P3]	DOCS/09 — Infrastructure architecture	Section 16
[P4]	DOCS/10 — Redis caching and memory	Section 16 , Section 17.1
[P5]	DOCS/11 — Microservices transition	Section 17.1
[P6]	DOCS/12 — Known gotchas	Section 13.5
[P7]	DOCS/14 — VPC networking	Section 17.1
[P8]	README.md — Project overview	Section 16

Table 32 — Project documents referenced.

D.2 External documentation

The following product documentation defines the behaviour of components this system depends on and was used to interpret the configuration surface. Version-specific behaviour should be confirmed against the vendor documentation current at the time of reading.

Reference	Subject
[E1]	LangGraph — state graphs, conditional edges and checkpoints
[E2]	NVIDIA NeMo Guardrails — Colang flows and rails configuration
[E3]	Portkey — gateway configurations, virtual keys, caching and fallback
[E4]	Qdrant — collections, distance metrics and the query interface
[E5]	Google Cloud Document AI — processor locations and synchronous request limits
[E6]	Google Cloud Vertex AI — text embedding models and quota
[E7]	RAGAS — metric definitions and judge configuration
[E8]	FlashRank — reranking models and local execution

Table 33 — External documentation.